IEEE 版权声明

© 2022 IEEE。允许个人使用此材料。所有其他用途，无论在当前还是未来的任何媒体中，均须获得 IEEE 的许可；这些用途包括将该材料转载/重新发布以用于广告或促销目的、创建新的集体作品、转售或再分发到服务器或列表，或在其他作品中重用本作品中任何受版权保护的组成部分。

This 论文已被接受发表在IEEE机器人与自动化快报（RA-L).

DOI: 10.1109/LRA.2022.3151396

IEEE Explore: https://ieeexplore.ieee.org/document/9714001

arXiv:2202.05481v2 [cs.RO] 2 Mar 2022

面向动态且稳健的腿式运动的控制策略与状态估计器同步训练

Gwanghyeon Ji、Juhyeok Mun、Hyeongjun Kim 和 Jemin Hwangbo*

摘要：本文提出了一种同时训练控制策略和状态估计器的运动训练框架。该框架由一个输出期望关节位置的策略网络和一个输出机器人状态估计（如机体线速度、足端高度和接触概率）的状态估计网络组成。我们利用快速仿真环境训练网络，并将训练好的网络迁移到真实机器人上。训练后的策略和状态估计器能够穿越多种地形，例如坡地、光滑平板和颠簸路面。我们还证明了学到的策略在普通平坦地面上能以最高***3.75 m/s***的速度奔跑，在摩擦系数为***0.22***的光滑平板上能以最高***3.54 m/s***的速度奔跑。



图1. 在摩擦系数为0.22的光滑平板上进行动态且稳健的运动。Mini Cheetah 以平均速度3.54米/秒通过该平板。

I. 引言

近年来，强化学习（RL）已成为腿式机器人最流行的控制方法之一。在四足机器人的动态运动技能学习方面，已取得显著进展。Hwangbo等人 [1] 为ANYmal机器人 [2] 训练了控制策略，用于在大扰动下保持平衡的同时实现稳健且高速的运动。在后续工作 [3] 和 [4]，中，经过RL训练的策略使四足机器人能够穿越各种具有挑战性的地形，如湿滑地面、植被和岩石地形。他们训练了一个编码器，该编码器压缩环境信息，并实现了有效的环境感知运动。此外，Peng等人 [5] 通过模仿记录的运动轨迹数据，重现了动物的敏捷动作。他们使Laikago机器人以中等速度行走和转弯。

诸如从跌倒中动态恢复 [1]，[6]，等更复杂的训练行为已有文献报道。这些复杂行为可以通过使用预训练专家网络和门控神经网络的单一框架组合而成，并在真实机器人上展现出敏捷而有效的运动 [7] 。此外，强化学习还可用于双足机器人攀爬楼梯 [8] ，或展现站立、行走和跑步等多种运动模式 [9]。

现有的四足运动控制方法依赖于准确估计的状态输入 [1]，[3]，[4]，[7]，[10]–[13]。然而，我们观察到现有的状态估计算法在具有挑战性的地形（如冰面和沙地）上会变得不可靠 [14]，[15]。此外，许多状态估计算法，例如Mini Cheetah机器人内置的算法，需要预先知道步态模式。大多数神经网络控制策略通常不提供此类信息，因为这些模式也是学习得到的。另外，接触状态可以通过专用的接触传感器或从模型中估计 [16]，，但前者计算成本高，后者在脚着地时容易遭受永久性损坏。因此，开发一个不依赖接触信息的控制框架是非常可取的。

此外，为了在具有挑战性的地形上盲态行走和奔跑，必须估计有关地形的信息。可以采用解析方法 [10] 来估计部分信息。或者，也可以利用训练好的神经网络和本体感觉状态历史 [3]，[4]来隐式地估计该信息。本体感觉状态历史对于估计机器人的内在状态和外在环境变量都很有用。然而，这种方法有两个主要缺点。第一，由于潜在向量不可解释，它们无法与需要状态信息的其他模块结合使用。第二，编码器训练会带来显著的计算开销。另一种方法是直接估计可观测的状态变量，例如地形角度。在我们提出的方法中，该信息被间接估计为地形到足部的距离。

为了解决现有方法的上述不足，我们提出了一种基于学习的状态估计网络，该网络与策略网络同时训练。我们使用Mini Cheetah机器人 [17]，（一种轻量且高动态的四足机器人）验证了所提方法的有效性。Kim等人 [11]

这项工作得到了三星电子旗下未来技术研发与孵化中心的支持，项目编号为 SRFC-IT2002-02。Mini Cheetah 机器人由麻省理工学院仿生机器人实验室和 Naver Labs 公司提供。

*所有作者均隶属于韩国大田广域市（邮编 34141）韩国科学技术院（KAIST）机械工程系的机器人与人工智能实验室。jhwangbo@kaist.ac.kr

报道了一种基于MPC的控制器，能使Mini Cheetah在跑步机上以高达3.7 m/s的速度奔跑，并在室外环境中[17]实现2.45 m/s的速度。我们在此报告了在各种室内和室外环境中以3.74 m/s进行的高度动态运动，以及在滑板、崎岖柏油路和山坡上稳健可靠的运动行为。

我们的主要贡献如下：

- 我们提出了一种简单的端到端运动学习框架，可同时训练控制策略和状态估计器。
- 利用训练后的网络，我们展示了在湿滑地形和斜坡上的动态运动能力。
- 我们分享了训练细节，例如动态随机化和课程学习，以便其他研究人员能够复现我们的工作。

II. 方法

我们的目标是开发一个基于强化学习的控制框架，使其能够跟踪给定的速度指令，该指令由前向和侧向的期望本体线速度以及期望偏航角速度组成。我们假设机器人配备了惯性测量单元（IMU）和关节编码器。

我们的控制框架概览如图2所示。该框架由三个不同的神经网络组成：估计器网络、评论家网络和行动者网络。估计器网络利用机载传感器估计控制所需的多个相关状态变量，并将其输入行动者网络，由行动者网络输出执行器指令。评论家网络有助于降低强化学习算法中策略梯度估计的方差。所有神经网络均在RaiSim [18]仿真环境中训练。

我们使用近端策略优化（PPO） [19] 来训练行动者网络和评论家网络，并使用监督学习来训练估计器网络。在RaiSim中采集一批轨迹后，我们分别使用对应的损失函数更新这三个网络。该过程像标准PPO一样重复，直到性能指标收敛。这两个网络的并行学习可确保策略网络能够适应估计器网络的性能特性。例如，当脚底接触打滑导致估计不可靠时，策略网络将使用不可靠的状态估计进行训练，从而表现出保守的行为。

本工作使用的机器人平台是 Mini Cheetah 机器人 [17] 。它体积紧凑、执行器强劲，使我们能够应对诸如在湿滑地形上进行高速运动等困难任务。此外，机器人操作的开源代码可在网上获取[1]，，其中包含一个高性能运动控制器 [11]。而且，其 IMU 传感器（Lord Corporation 制造的 3DMGX5-AHRS）不仅提供线性加速度和角速度，还提供估计姿态，该估计基于扩展卡尔曼滤波器。与 [17] 中介绍的原始版本相比，我们使用的 Mini Cheetah 版本重 1.8 千克，长 1 厘米。我们在真实机器人上的实现代码可以在网上找到[2]。

A. 仿真训练

该策略在 800 个不同环境的平坦地形上进行训练，以高效地收集样本。在每个环境中，机器人均以表 I 所示的高度随机的初始状态进行初始化。这有助于机器人从意外外部干扰（如与人类的交互或地形参数突变）中恢复。有 25% 的概率，机器人会以上一回合的最终状态初始化，从而学会应对真实世界中速度指令的突然变化。为进一步改进，我们还会在随机生成崎岖平地或坡度最高达 $\pm 10^\circ$ 的环境中训练模型。崎岖地形使用具有以下参数的 Perlin 噪声生成：分形倍频程 =5，分形空隙度 = 3.0，分形增益 = 0.45，z 轴缩放 =min(0.21, $0.21 \cdot t^{-1}$)，其中 t 为迭代次数。

表 I 初始状态噪声

状态	噪声
四元数	$U^4(-0.2,0.2)$，然后归一化
关节位置	$U^{12}(-0.2,0.2)$ 弧度
关节速度	$U^{12}(-2.5,2.5)$ rad/s
X 线性速度	$U^1(-1,1)$ m/s
YZ 线性速度	$U^2(-0.5,0.5)$ m/s
角速度	$U^3(-0.7,0.7)$ rad/s

为了更有效地训练策略，我们设置了一个课程，其中 x 方向（即前进/后退方向）的速度指令会随每次 PPO 迭代逐渐增加。在训练早期，x 方向的线速度指令从 $U^1(-0.5,1.0)$m/s 中均匀采样。该范围逐渐扩大至 $U^1(-1.75,3.5)$m/s，最大前进指令按如下方式给出：

$$V_{x,max} = 1 + \frac{2.5}{1 + exp(-0.002 \cdot (t - 1000))}, \quad (1)$$

其中 t 是训练迭代次数。然后选择 10% 的轨迹赋予零速度指令，以学习静止站立行为。

我们在表II中定义了奖励函数及其系数。奖励函数的设计目标有两个：遵循给定指令，以及以高效自然的方式奔跑。线速度和角速度奖励与前一目标相关，其他奖励则用于后一目标。大多数奖励函数是参考 [1]设计的。其中，足部离地间隙奖励对于成功的仿真到现实迁移至关重要，因为在某些情况下，足部相对于地面的位置和地形几何形状可能不确定

[1]https://github.com/mit-biomimetics/Cheetah-Software

[2]https://github.com/karlji1021/Cheetah-Software

情况下。足部离地间隙奖励中的平方根函数旨在当指令速度过低时增强其对策略的影响。腾空时间奖励用于控制摆动-支撑时序并生成静止站立动作。

表 II 奖励函数

奖励	表达式				
线速度	$r_v = k_v \exp(-\|\|cmd_{vxy} - v_{xy}\|\|)$				
角速度	$r_\omega = k_\omega \exp(-1.5(cmd_{\omega z} - \omega_z)^2)$				
空中时间					
$r_{air,i}$	if stance cmd: $k_{acli}\, p(T_{s,i} - T_{a,i}, -0.3, 0.3)$ 否则：如果 $T_{max,i} < 0.25$: $k_{amin}(\max(T_{s,i}, T_{a,i}), 0.2)$ 否则：0				
足部滑移	$r_{slip,i} = k_{slip} C_{f,i} \|\|v_{f,xy,i}\|\|^2$				
足部离地间隙	$r_{cl,i} = k_{cl}(w_{pf,z,i} - w_{pdes\,f,z})^2 \|\|v_{f,xy,i}\|\|^{0.5}$				
姿态	$r_{ori} = k_{ori}(angle(\phi_{body,z}, \phi_{world,z}))^2$				
关节力矩	$r_\tau = k_\tau \|\|\tau\|\|^2$				
关节位置	$r_q = k_q \|\|q_t - q_{nominal}\|\|^2$				
关节速度	$r_{\dot{q}} = k_{\dot{q}} \|\|\dot{q}_t\|\|^2$				
关节加速度	$r_{\ddot{q}} = k_{\ddot{q}} \|\|\dot{q}_t - \dot{q}_{t-1}\|\|^2$				
动作平滑度 1	$r_{s1} = k_{s1} \|\|q_{dest} - q_{dest-1}\|\|^2$				
动作平滑度 2	$r_{s2} = k_{s2} \|\|q_{dest} - 2q_{dest-1} + q_{dest-2}\|\|^2$				
基础运动	$r_{base} = k_{base}(0.8 v_z^2 + 0.2\|\omega_x\| + 0.2\|\omega_y\|)$				
奖励系数					
k_v	3.0	k_{cl}	-15.0	$k_{\dot{q}}$	-6e-4
k_ω	3.0	k_{ori}	-3.0	$k_{\ddot{q}}$	-0.02
k_a	0.3	k_τ	-6e-4	k_{s1}, k_{s2}	-2.5, -1.2
k_{slip}	-0.08	k_q	-0.75	知识库	-1.5

在表 II 中，cmd 是 command 的缩写，i 是脚的索引。$T_{a,i}$ 和 $T_{s,i}$ 分别表示自上次起飞以来的时间和自上次触地以来的时间，并在每次状态转换发生时初始化为零。在脚滑奖励中，$C_{f,i}$ 是每只脚的接触状态。在脚离地间隙奖励中，$p^{des}_{f,z}$ 是期望的脚高度，设置为 0.09 m。我们将正奖励总和定义为 $r^{pos} = r_v + r_\omega + \sum_{i=0}^{3} r_{air,i}$，将负奖励总和定义为 $r^{neg} = \sum_{i=0}^{3}(r_{slip,i} + r_{cl,i}) + r_{ori} + r_\tau + r_q + r_{\dot{q}} + r_{\ddot{q}} + r_{s1} + r_{s2} + r_{base}$。总奖励定义为

$$r_{tot} = r_{pos} \cdot exp(0.2 r_{neg}) \tag{2}$$

这种奖励函数形式可确保最终奖励始终为正，并抑制策略选择提前终止。

每当机器人身体（膝盖和脚除外）接触环境时，回合终止，机器人会获得 -10 的奖励作为惩罚。因此，策略被训练以减少不必要的碰撞。

B. 网络架构

我们的神经网络结构包含3个组件：动作网络（actor）、评价网络（critic）和估计网络（estimator）。它们都被设计为多层感知器（MLP）网络，其中动作网络和评价网络采用 [512×256×64] 结构，估计网络采用 [256×128] 结构。MLP是最简单的神经网络结构，在计算上比其他基于记忆的网络（如RNN）更高效。我们以LSTM的形式训练策略，但但未能发现性能上的显著差异。动作网络将观测映射为动作，评价网络 [20] 估计当前状态的价值。估计网络用于估计机器人的状态，例如基座线速度。这些值以观测o_t 作为输入进行估计，并馈送给动作网络。估计网络利用仿真数据进行监督学习训练。策略和估计器以100 Hz的频率同步运行。网络结构如图2所示。

我们的系统将传感器数据作为输入，并为每个执行器输出期望的关节位置。我们的框架仍然使用机体坐标系中表示的重力向量的解析估计，因为该向量由IMU传感器计算得到。此外，姿态估计算法简单且可靠。关节速度由电机控制器通过对关节位置应用有限差分法计算得到。

观测元组定义如下。

$$o_t = (\phi, \omega, q, \dot{q}, q^{des}_{t-1}, q^{des}_{t-2}, Q_{hist}, \dot{Q}_{hist}, b\,p_f, cmd) \tag{3}$$

其中，ϕ 和 ω 分别为基座姿态和角速度，q和 $\dot{q}$分别为关节位置和速度，q^{des}_{t-1} 和q^{des}_{t-2} 为前两个时间步的期望关节位置目标，Q_{hist} 和Q_{hist} 分别为关节位置误差历史和关节速度历史，b p f 为足端在机体坐标系中相对于质心的笛卡尔位置，cmd 为给定的速度指令。笛卡尔足端位置用于观察足端所在位置，已知有助于针对复杂系统 [21]训练策略。在本研究中，关节状态历史选取在 t −0.02s、t−0.04s 和 t−0.06s 处。为了实现稳定的学习与控制，所有观测变量都被归一化为均值为0.0、标准差为1.0。出于同样的原因，策略输出先乘以一个标称值，然后加上标称关节位置，以获得期望的关节目标分布。该关系表示如下。

$$q^{des}_t = q_{nominal} + \sigma_a a_t,$$

其中 $q_{nominal}$ 是标称关节位置，与站立构型相同，$\sigma_a = 0.1$ 是预定义的动作缩放因子，a_t 是策略网络的输出。期望关节位置以 100Hz 的频率计算，并在真实机器人上以 40kHz 的频率由具有 K_p=17 N·m·rad^{-1} 和 K_d=0.4 N·m·s·rad^{-1} 的 PD 控制器模块转换为关节力矩。

估计器网络的设计目的是在不使用专用估计算法的情况下预测机器人的状态。本文对线速度、足端高度和接触概率进行估计。线速度估计对于以下速度指令至关重要。由于不再需要复杂的状态估计算法，在机器人上的实现变得更加简单。它还有一个优点，即控制器对状态估计器不可避免的误差具有鲁棒性。如 [15]，所示，在存在较大误差的环境下，线速度的估计是

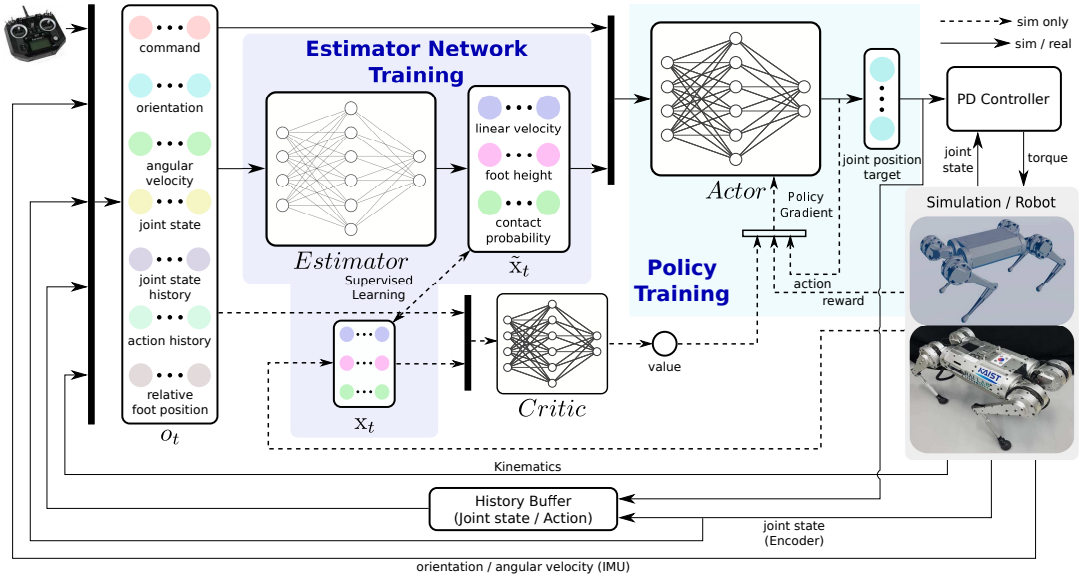


Fig. 2. An overall control diagram and a proposed training framework for concurrent training of a control policy and a state estimator are shown. The estimator network takes sensor data o_t as an input and outputs state variables, which are the base linear velocity, foot height, and contact probabilities. These estimated states are fed into the policy network together with the observation o_t. The estimator network is trained with supervised learning to reduce MSE between the estimated robot states and their corresponding true values obtained from simulation. An actor network is a policy which produces desired joint positions based on the state estimates. Both the critic and actor are trained using the PPO algorithm.

vulnerable to foot slips. Our end-to-end neural network structure avoids such a challenge in two ways. First, the estimator network is trained in environments where the feet slip often. Therefore, it can still produce a reasonable estimate of the linear velocity using other sensor information and previous observations. Second, the policy network is trained with imperfect information such that it is aware of possible slippages.

The idea behind learning foot height and contact probability is to achieve the sufficient foot clearance. Due to the wide range of velocity command and the clearance reward that penalizes high speed, the foot clearances become smaller at low speeds. Foot clearance plays an important role in a sim-to-real transfer because insufficient foot clearance might lead to an early foot landing or tripping while running. We discovered that the reward term alone is not sufficient to learn to maintain sufficient foot clearances. Our solution to this problem is to estimate the foot clearances and feed them directly to the policy network. We note that a learning-based estimator is capable of approximately estimating the foot clearance from the observation. First, foot contact states are obtainable from joint position errors. Second, a terrain slope becomes observable from the foot contact states, orientation, and joint positions. Therefore, as the slope is observable, the estimator network can compute the foot height under the assumption that the terrain is even.

C. Dynamics Randomization

Dynamics randomization is important for a successful sim-to-real transfer of policies trained in simulation [22]. In our case, the robot controller learned without dynamics randomization exhibits shaky motions when deployed on the real robot. It comes from the fact that kinematic and dynamic parameters such as leg length, actuator positions, and center of mass, are not exactly the same as those used in the simulation. Consequently, this reality gap often makes the robot unable to reach sufficient performance. To eliminate this gap, we randomize several components as follows:

- observation noise
- motor frictions
- PD controller gains
- foot positions and collision geometry
- ground friction

These parameters are randomized at the start of each episode or iteration.

The observation noise is added during the training phase in the simulation. On the real robot, joint velocity measurement comes from numerical differentiation of the joint positions, which causes errors in the joint velocity observation. Moreover, fluctuation in the logging frequency might lead to a failure in updating the velocity values for a single time step. Such an event corresponds to a delay of 2 milliseconds. Therefore, the joint position and velocity measurements in simulation are randomized to reflect the true distribution of the measurements; they are sampled from $U^{12}(-0.05,\ 0.05)$ rad and $U^{12}(-0.5,\ 0.5)$ rad/s, respectively. For the same reason, a uniformly distributed noise $U^4(-0.03,\ 0.03)$, $U^4(-0.03,\ 0.03)$ m, and $U^3(-0.1,\ 0.1)$ rad/s are added to the base orientation, foot position, and base angular velocity, respectively.

Motor friction is randomized to reflect the differences between actuator units. We chose a conservative range of $U^1(0,\ 0.3)$ N·m for the hip abduction/adduction (HAA) and hip flexion/extension (HFE), and $U^1(0.1,\ 0.7)$ N·m for the knee flexion/extension (KFE). Their measured friction values on the real robot are 0.2, 0.2, and 0.5 N·m for HAA,

HFE, and KFE, respectively. The KFE joints have higher friction because they have an extra belt transmission. The PD controller gains are randomized to mitigate the effects of motor friction and damping. We added a uniform noise of $U^1(-2,\ 2)\,\mathrm{N{\cdot}m{\cdot}rad^{-1}}$, and $U^1(-0.1,\ 0.1)\,\mathrm{N{\cdot}m{\cdot}s{\cdot}rad^{-1}}$ for the position and velocity gains, respectively.

The foot position and collision geometry are randomized to reduce both effects of measurement errors and the deformation of the rubber feet. The foot position observations are disturbed with a uniform noise of $U^1(-10,\ 10)$ mm in the longitudinal direction, $U^1(-5,\ 5)$ mm in the lateral direction, and $U^1(-20,\ 20)$ mm in the leg length direction. These noises are added to the measured foot positions. The foot sphere radii are randomized to $U^1(6,\ 10)$ mm.

Finally, the ground friction was randomized to $U^1(0.4,\ 1.0)$. Owing to this randomization, the robot can run not only on very slippery ground but also on the ground with very high friction like asphalt.

III. RESULTS

Part of the results in this section can also be found in the accompanying video.

A. Controller Descriptions

To evaluate the performance of the proposed network structures and analyze how each component affects different performance metrics, we test the following settings:

- *Implicit*: As a baseline, the explicit state estimator in our proposed framework is substituted with an implicit estimator as in [3], [4].
- *Sequential*: In phase 1, a policy is trained with the ground truth robot states. In phase 2, a state estimator was trained with the final policy of phase 1. For implementation, the estimator replaces the ground truth input.
- *Built-in MPC*: The MPC controller described in [17]
- *RL-LKF*: An RL policy in a single MLP form trained with the linear velocity data from the simulator and uses an LKF state estimator on the real robot.
- *Concurrent*: Our proposed control framework trained on relatively flat ground
- *Concurrent+Slope*: Our proposed control framework trained on randomly generated slopes.

In addition to the above models, we also created four other network models by excluding one of the three estimated states or all of them (i.e., *w/o Linvel Estimator*, *w/o FootHeight Estimator*, *w/o Contact Estimator*, *w/o Estimator*) for ablation studies in simulation.

For comparison of the explicit and implicit estimators, we trained the *Implicit* model. The implicit estimator follows the framework in [4]. During the phase 1, the encoder takes the observation $o_t \backslash (Q_{hist}, \dot{Q}_{hist})$, linear velocity, foot height, and foot contact as an input. For the adaptation module, the history length of 20 is used and the network consists of 3 layers of 1D CNN. The output dimension of the encoder and adaptation module is 11. All the other settings are the same as our framework. The total training time is 7 hours for phases 1 and 2 altogether.

B. Evaluation of the Performance in Simulation

1) Learning Performance: First, we compared the performance of the learned controllers (i.e., *Concurrent*, *w/o Linvel Estimator*, *w/o FootHeight Estimator*, *w/o Contact Estimator*, and *w/o Estimator*) in simulation. Each network structure is trained four times and their average learning curves are shown in Fig. 3. All the models were trained until they converged to a stable expected return. After 2500 iterations, which consumed about 800 million samples and 4 hours of training in real-time, they all converged to a stable value. The rewards of the trained models are summarized in the table III.

As shown in Fig. 3 and the table III, the *Concurrent* model converges to the highest rewards while the *w/o Estimator* and *Implicit* models converge to the lowest. Out of the three estimated states, linear velocity estimation plays the most important role in improving the policy. Omitting the linear velocity estimation leads to a significant drop in metrics: the total, linear velocity, and foot clearance rewards. This result proves that the linear velocity is crucial for learning the locomotion of legged robots.

Another noticeable improvement comes from the foot height estimation. Explicitly estimating the foot height improves the foot clearance of the robot, resulting in a higher foot clearance reward. The effectiveness of the higher foot clearance will be discussed in the *Locomotion on rough terrains* section.

Foot contact probability estimation makes the least impact on the final performance, but it stabilizes and accelerates learning processes as shown in the total reward graph in Fig. 3.

2) Tracking and Estimation Error: For further investigation on the impact of the estimator network, we tested the *Concurrent*, *w/o Estimator*, and *Implicit* models in simulation. All models were given the same random commands every 20 seconds and for 10 minutes in total while the friction coefficient of the flat ground is kept at 0.6. The performance was only measured for the steady-state errors. The velocity commands are sampled from $U^1(-1.75,\ 3.5)\,\mathrm{m/s}$, $U^1(-1,\ 1)\,\mathrm{m/s}$, and $U^1(-2,\ 2)\,\mathrm{rad/s}$, for V_x, V_y, and ω, respectively.

The result is shown in the table IV. The *Concurrent* model has the smallest RMS errors for following the given linear velocity. It means that the estimated states help the robot to stabilize itself. Furthermore, the tracking errors of the *Implicit* model are on a similar level to that of *w/o Estimator*. From this fact, we suggest that tracking performance does not benefit a lot from utilizing the implicit estimator. Models with the implicit estimator having a latent vector of sizes 8 and 20 showed worse performance than the presented one.

Interestingly, our concurrently trained model performs better than a sequentially trained model. From the ta-

TABLE III

ABLATION STUDY FOR THE ESTIMATOR:

REWARD OF THE LEARNED MODELS

Model	Reward		
	Total	Linear Velocity	Foot Clearance
Concurrent	4.7212	**2.7595**	**-0.1338**
w/o LinVel Estimator	4.5555	2.6643	-0.1432
w/o FootHeight Estimator	**4.7293**	**2.7625**	-0.1504
w/o Contact Estimator	4.7125	2.7543	-0.1382
w/o Estimator	4.4577	2.6284	-0.1565
Implicit	4.439	2.611	-0.1447

ble IV, the *Sequential* model has slight performance degradation. We hypothesize that this is because the policy trained with a state estimator tends to avoid states where the state estimator becomes unreliable. This problem can be easily solved by training them concurrently as we proposed. Note also that the concurrent training requires only one training dataset, which is more efficient than the sequential training.

3) Locomotion on rough terrains: We investigated the effectiveness of the foot clearance of the learned models. The foot clearance should be sufficiently high for traversing over the rough terrains. Therefore, in this experiment, we compare the average time to fall on the rough terrains with z-scale of 0.525. Commands are sampled from U^1(-1, 1) m/s, and U^1(-1, 1) rad/s where the foot clearance is relatively small.

From the table IV, the *Concurrent* model shows an overwhelming performance compared to the other models. It is robust against a fall due to increased foot clearance. On the other hand, an implicit estimator and a single policy do not have sufficient foot clearance. Eventually, they are easy to fall and have worse locomotion performance. In conclusion, we suggest that the explicit estimation of the foot height significantly contributes to locomotion performance.

4) Computational Cost: The trained estimator has computational benefits over analytical state estimators. Using a single core of Ryzen9 5950x, the estimator network takes 7 μs for a forward pass, while the linear Kalman filter on the Mini Cheetah consumes 34 μs. Also, the implicit estimator with 20 history inputs takes 20 μs which is about three times longer than the simple explicit estimator.

TABLE IV

TRACKING AND ESTIMATION ERROR

& LOCOMOTION ON ROUGH TERRAINS

Task	Model	V_x [m/s]	V_y [m/s]	ω_z [rad/s]
Command Following	*Concurrent*	**0.1112**	**0.0708**	0.1222
	w/o Estimator	0.1725	0.0959	**0.1137**
	Implicit	0.1679	0.1233	0.1379
	Sequential	0.1358	0.0996	0.1201
Estimation Error	*Concurrent*	**0.0243**	**0.0199**	-
	Sequential	0.06	0.036	-
		Average time to fall [sec]		
Rough Terrain	*Concurrent*	**85.7**		
	w/o Estimator	25.0		
	Implicit	30.0		
	Sequential	75.0		

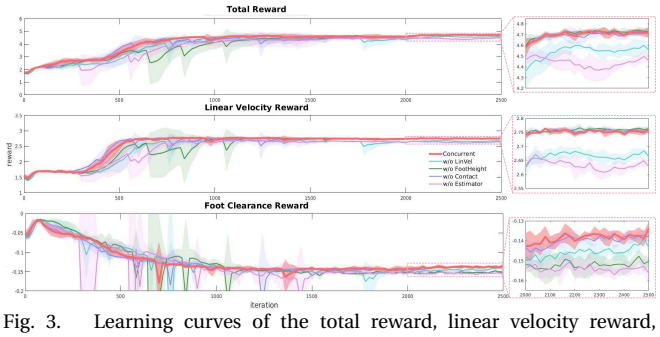


Fig. 3. Learning curves of the total reward, linear velocity reward, and foot clearance reward are shown. The linear velocity, foot height, and contact probability are estimated. The *Concurrent* model has the highest performance and learning stability, while the model without an estimator (*w/o Estimator*) has the lowest performance.

C. Evaluation of the Performance on the Real Robot

We evaluated the performance of controllers on the real robot in terms of command following, state estimation, the maximum running speed, the maximum traversable slope angle, and foot clearance. The *Concurrent+Slope* model requires 5000 iterations for convergence due to the challenging terrains. The summary of the performance is shown in the table V.

1) Network Implementation on the Real Robot: For the experiments in the real environments, we compared the five aforementioned control models: *Built-in MPC*, *RL-LKF*, *Concurrent*, and *Concurrent+Slope*. As described in the *Controller Description* section, the *RL-LKF* model requires an LKF state estimator. However, we could not use the built-in contact estimator as it assumes a periodic gait schedule, while our learned policy inherently changes gait patterns over speeds. Therefore, when estimating the contact state of the *RL-LKF* model for the LKF, we used the proprioceptive touchdown detection method described in [23]. When the difference between the KFE joint position and its desired position is over the threshold of -0.4 rad, we assume that the leg is in contact.

2) Command Following and State Estimation: In the real environments, each controller was tested on normal and slippery ground with a step velocity command of 1.0 m/s, 0.8 m/s, and 2.0 rad/s for linear velocities in x and y directions, and a yaw rate, respectively. The commands continued for 1 second and the robot started from zero commands. The slippery plate covered with boric acid powder has a friction coefficient of 0.22, which is much lower than that of the training environments. The results are shown in the table V. In some cases, the *Built-in MPC* controller fell and those instances are marked with '-' in the table. On the other hand, RL policies performed robustly against all sudden step inputs in this experiment. The *Concurrent* model has the best tracking performance, while the *Concurrent+Slope* model has a slightly higher error possibly due to the fact that the *Concurrent* model is overfitted to simpler terrains.

We also recorded the estimation data from the LKF while the *Concurrent* and *Concurrent+Slope* models are

TABLE V

PERFORMANCE OF CONTROLLERS ON THE REAL ROBOT

Task	**Terrain** (friction)	**Command**	**Model**				
			Built-in MPC	*RL-LKF*	*w/o Estimator*	*Concurrent*	*Concurrent+Slope*
Command Following : Tracking Error	Normal ($\mu = 0.6$-0.88)	V_x [m/s]	0.3041	0.2744	0.2635	**0.2387**	0.2488
		V_y [m/s]	-	0.2031	0.2334	**0.1722**	0.1817
		ω_z [rad/s]	0.462	0.2857	0.2415	0.2427	**0.2395**
	Slippery ($\mu = 0.22$)	V_x [m/s]	-	0.3255	0.2888	0.2874	**0.2532**
		V_y [m/s]	-	0.285	0.2617	**0.2183**	0.2446
		ω_z [rad/s]	0.525	0.3229	0.2709	**0.2504**	0.2654
State Estimation : Estimation Error	Normal ($\mu = 0.6$-0.88)	V_x [m/s]	0.1869	0.0808	**0.0653**	0.0783 (0.1069)	0.0852 (0.0946)
		V_y [m/s]	-	0.1115	0.0666	0.1227 (0.0793)	**0.0557** (0.078)
	Slippery ($\mu = 0.22$)	V_x [m/s]	-	0.1575	0.0741	**0.0533** (0.1209)	0.1168 (0.1029)
		V_y [m/s]	-	0.0623	0.0612	**0.0555** (0.0848)	0.0833 (0.1061)
Maximum Average Speed	Normal ($\mu = 0.6$-0.88) [m/s]		1.72	2.18	3.20	3.33	**3.75**
	Slippery ($\mu = 0.22$) [m/s]		1.34	2.19	3.14	3.25	**3.54**
Maximum Slope	Normal ($\mu = 0.6$-0.88) / Slippery ($\mu = 0.22$) [°]		12.4	12.4	9.6	12.4	**19.1** / 9.0
Maximum Foot Height	Normal [cm]		**5**	2	2	3	3

running. The errors from the estimator networks are written in the parentheses. We note that estimation error does not necessarily lead to a deterioration in tracking performance. We suppose that other observation inputs, such as joint state history, mitigate the effects of the estimation errors so that the concurrent learning framework becomes robust against these errors.

3) High-Speed Locomotion: The *Concurrent+Slope* model has the highest maximum speed. We tested each controller repeatedly until the robot fell. The maximum outdoor speed of our model, 3.75 m/s, is comparable to the one reported by Kim et al. [11], who report outdoor speed of over 1.7 m/s and treadmill speed of 3.7 m/s.

Our *Concurrent+Slope* controller is capable of running at 3.54 m/s on a slippery plate with $\mu = 0.22$. When large foot slippages occurred, the robot recovered quickly, even when the robot is running near the maximum speed. If the command is suddenly set to zero while running, the robot makes a stable pose to stop quickly. We assume that this high performance is achieved owing to two factors: the policy trained on low friction terrains and its independence on a state estimation algorithm. Because our proposed framework is trained to be aware of possible foot slippages, it can be robust against estimation errors.

On both normal and slippery ground, the *Concurrent* model exhibits better performance than the *Built-in MPC* and *RL-LKF* models. The *Built-in MPC* model could not reach speeds over 1.7 m/s on the normal ground and was easy to fall on the slippery ground at speed under 1.3 m/s. *RL-LKF* model also runs at rather conservative speeds lower than 2.2 m/s. On the other hand, all the RL controllers are able to run consistently on all tested terrains as they are trained on terrains with the various friction coefficients.

4) Locomotion on Hills: Training a policy on slopes with random angles and friction coefficients significantly improves climbing performance. The *Concurrent+Slope* model is capable of climbing a normal hill up to 19.1°, which is steeper than slope angles of the training environments, ±10°. Also, we note that *Concurrent+Slope* model can walk up a slippery hill up to 9.0°. Although the feet of the robot are slipping on it, the robot managed to climb up the slope by pushing the ground with adequate force. This behavior is partially learned in simulation, but the robot adapts its motion for the much more slippery slope of the friction coefficient of 0.22. For outdoor environments, we demonstrate the performance of our policy on a bumpy and hilly asphalt road. Please refer to the supplementary video for the demonstration.

The other controllers could not climb up a normal hill with angles steeper than 12.4°. Because the RL controllers except for the *Concurrent+Slope* model are trained only on nearly flat terrains, they have unsatisfactory climbing performance. Although the other controllers' maximum traversable slope angles are on a similar level, they display different locomotion characteristics. The *Built-in MPC* model has higher foot clearances, but its non-stopping gait makes the robot unstable. The other RL models show relatively lower foot clearances, while their standing-still behavior significantly improves the stability of the robot.

5) Foot Clearance: We evaluated the foot clearance of each controller while the robot is running at 1.0 m/s. The maximum foot clearance is shown in the table V. We could recognize that foot clearance of the *Concurrent* and *Concurrent+Slope* models on the real robot is higher than the models without the estimator network, *RL-LKF* and *w/o Estimator*. As shown in the simulation test, lower foot clearance hinders stable locomotion on highly uneven terrains. This issue remains equally problematic on the real robot. We discovered that the *RL-LKF* and *w/o Estimator* models exhibit lower foot clearance at low speeds. Therefore, we suggest that explicit estimation of the foot clearance is effective for improving the performance.

6) Contact Estimation: Our proposed estimator network outputs contact probability for each foot. In Fig. 4, the contact state probability of the front right foot is drawn with the real contact state, KFE joint position error and joint velocity. The estimated contacts are shifted by 0.04 seconds from the actual contacts, which corresponds to 3 control steps excluding 0.01 seconds of communication

delay. This is because there is a reality gap due to the compliance of the chain and the rubber feet of the real robot. In addition, the joint history inputs are sparsely sampled with 0.02-second intervals and it introduces further delay in detection. The estimator network detects a contact when the joint abruptly stops by impacts with the ground. The diagram also justifies the threshold of -0.4 rad of joint position error for the contact estimation used for LKF.

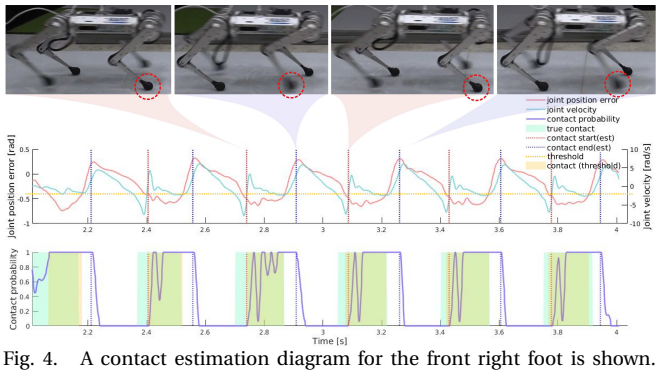


Fig. 4. A contact estimation diagram for the front right foot is shown. KFE joint position error, joint velocity, and contact state are drawn. Contact starts with an abrupt change of the joint velocity by impact with the ground and ends with upward joint motion.

IV. CONCLUSION

We presented a framework for concurrent training of a control policy and a state estimator. This framework requires neither an advanced control algorithm nor an accurate state estimation algorithm. Therefore, it requires significantly less effort for implementation on the real-legged system. Furthermore, our concurrent training method outperforms implicit estimation methods and a sequential training method in many aspects such as command tracking, robustness on rough terrain, and training time. To the best of our knowledge, our record is the fastest reported legged locomotion using reinforcement learning. The robot is also able to stably run on a slippery plate even under foot slippages. Although foot slippages often compromise the quality of the state estimation, the concurrently trained policy is robust against such issues.

The proposed learning-based state estimation can be useful without the control policy in many applications. It can provide reliable state estimates for motion analysis. Furthermore, we expect that the interpretable state outputs from our proposed network can be useful in conjunction with other controllers, such as MPC-based ones.

REFERENCES

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.

[2] M. Hutter, C. Gehring, D. Jud, A. Lauber, C. D. Bellicoso, V. Tsounis, J. Hwangbo, K. Bodie, P. Fankhauser, M. Bloesch, R. Diethelm, S. Bachmann, A. Melzer, and M. Hoepflinger, "Anymal - a highly mobile and dynamic quadrupedal robot," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 38–44.

[3] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, 2020.

[4] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems Conference (RSS 2021)*, 2021.

[5] X. B. Peng, E. Coumans, T. Zhang, T.-W. E. Lee, J. Tan, and S. Levine, "Learning agile robotic locomotion skills by imitating animals," in *Robotics: Science and Systems (RSS 2020)*, 07 2020.

[6] J. Lee, J. Hwangbo, and M. Hutter, "Robust recovery controller for a quadrupedal robot using deep reinforcement learning," *arXiv preprint arXiv:1901.07517*, 2019.

[7] C. Yang, K. Yuan, Q. Zhu, W. Yu, and Z. Li, "Multi-expert learning of adaptive legged locomotion," *Science Robotics*, vol. 5, no. 49, 2020.

[8] J. Siekmann, K. Green, J. Warila, A. Fern, and J. Hurst, "Blind bipedal stair traversal via sim-to-real reinforcement learning," in *Robotics: Science and Systems Conference (RSS 2021)*, 2021.

[9] J. Siekmann, Y. Godse, A. Fern, and J. Hurst, "Sim-to-real learning of all common bipedal gaits via periodic reward composition," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[10] C. Gehring, C. D. Bellicoso, S. Coros, M. Bloesch, P. Fankhauser, M. Hutter, and R. Siegwart, "Dynamic trotting on slopes for quadrupedal robots," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 5129–5135.

[11] D. Kim, J. D. Carlo, B. Katz, G. Bledt, and S. Kim, "Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control," in *arXiv preprint arXiv:1909.06586v1)*, 2019.

[12] S. Hong, J.-H. Kim, and H.-W. Park, "Real-time constrained nonlinear model predictive control on so(3) for dynamic legged locomotion," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 3982–3989.

[13] Y. Ding, A. Pandala, C. Li, Y.-H. Shin, and H.-W. Park, "Representation-free model predictive control for dynamic motions in quadrupeds," *IEEE Transactions on Robotics*, vol. 37, no. 4, pp. 1154–1171, 2021.

[14] R. Hartley, M. Ghaffari, R. M. Eustice, and J. W. Grizzle, "Contact-aided invariant extended kalman filtering for robot state estimation," *The International Journal of Robotics Research*, vol. 39, no. 4, pp. 402–430, 2020.

[15] J.-H. Kim, S. Hong, G. Ji, S. Jeon, J. Hwangbo, J.-H. Oh, and H.-W. Park, "Legged robot state estimation with dynamic contact event information," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 6733–6740, 2021.

[16] J. Hwangbo, C. D. Bellicoso, P. Fankhauser, and M. Hutter, "Probabilistic foot contact estimation by fusing information from dynamics and differential/forward kinematics," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 3872–3878.

[17] B. Katz, J. D. Carlo, and S. Kim, "Mini cheetah: A platform for pushing the limits of dynamic quadruped control," in *2019 IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6295–6301.

[18] J. Hwangbo, J. Lee, and M. Hutter, "Per-contact iteration method for solving contact dynamics," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 895–902, 2018.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017.

[20] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018.

[21] D. Reda, T. Tao, and M. van de Panne, "Learning to locomote: Understanding how environment design matters for deep reinforcement learning," in *Motion, Interaction and Games*, ser. MIG '20. New York, NY, USA: Association for Computing Machinery, 2020.

[22] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[23] D. J. Hyun, J. Lee, S. Park, and S. Kim, "Implementation of trot-to-gallop transition and subsequent gallop on the mit cheetah i," *The International Journal of Robotics Research*, vol. 35, no. 13, pp. 1627–1650, 2016.